👤 PESSOA 1 — AUTH SERVICE (login)

🎯 Objetivo:

Gerenciar login e gerar token

🔧 Passos

1. Instalar dependências

```
npm install jsonwebtoken bcryptjs
```

---

2. Criar usuário fake (inicialmente)

```
// src/models/user.js
module.exports = [
  {
    id: 1,
    email: "admin@email.com",
    password: "123456",
    role: "ADMIN"
  }
];
```

---

3. Criar login

```
// src/controllers/authController.js
const jwt = require("jsonwebtoken");
const users = require("../models/user");

exports.login = (req, res) => {
  const { email, password } = req.body;

  const user = users.find(u => u.email === email);

  if (!user || user.password !== password) {
    return res.status(401).json({ error: "Invalid credentials" });
  }

  const token = jwt.sign(user, "secret");

  res.json({ token });
};
```

---

4. Rota

```
// src/routes/authRoutes.js
const express = require("express");
const router = express.Router();
const controller = require("../controllers/authController");

router.post("/login", controller.login);

module.exports = router;
```

PESSOA 2 — USER SERVICE

🎯 Objetivo:

Cadastrar usuários

Endpoint:

```
POST /users
```

Lógica:

```
let users = [];

exports.createUser = (req, res) => {
  const user = { id: Date.now(), ...req.body };
  users.push(user);
  res.json(user);
};
```

👤 PESSOA 3 — COLLECTION SERVICE ⭐

🎯 Objetivo:

Cadastrar ponto de coleta

Endpoint:

```
POST /collection-points
```

Lógica:

```
let points = [];

exports.createPoint = async (req, res) => {
  const point = {
    id: Date.now(),
    ...req.body,
    status: "PENDING"
  };

  points.push(point);

  // chama notification-service
  await fetch("http://localhost:3005/notify", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(point)
  });

  res.json(point);
};
```

👤 PESSOA 4 — ADMIN SERVICE

🎯 Objetivo:

Aprovar ou reprovar pontos

Endpoint:

```
PATCH /approve/:id
```

Lógica:

```
exports.approve = async (req, res) => {
  const { id } = req.params;

  await fetch(`http://localhost:3003/collection-points/${id}`, {
    method: "PATCH",
    body: JSON.stringify({ status: "APPROVED" }),
    headers: { "Content-Type": "application/json" }
  });

  res.json({ message: "Aprovado" });
};
```

👤 PESSOA 5 — NOTIFICATION SERVICE

🎯 Objetivo:

Receber evento e notificar

Endpoint:

```
POST /notify
```

Lógica (simples):

```
exports.notify = (req, res) => {
  console.log("📢 Novo ponto cadastrado!");
  console.log(req.body);

  res.json({ message: "Notificação enviada" });
};
```

🔌 PORTAS (IMPORTANTE)

Cada serviço roda em uma porta:

Serviço	Porta
auth	3001
user	3002
collection	3003
admin	3004
notification	3005

COMO RODAR

Cada pessoa:

```
node src/app.js
```

🧪 TESTE COMPLETO (PASSO A PASSO)

1. Criar usuário
2. Criar ponto:

```
{
  "name": "Mercado X",
  "address": "Rua Y"
}
```

3. Ver console do notification-service:

```
📢 Novo ponto cadastrado!
```

4. Admin aprova

💡 MELHORIAS DEPOIS (SE DER TEMPO)

- banco de dados (MongoDB)
- autenticação real
- validação
- Docker
- filas (RabbitMQ)

🚀 RESUMO FINAL

Cada integrante:

- cria seu serviço separado
- roda em porta diferente
- comunica via HTTP
- segue estrutura padrão

Se quiser, posso te entregar:

☑ um projeto pronto base (zip mental 😄)
☑ ou montar com vocês tipo “passo 2: agora roda isso”
☑ ou revisar o código de vocês

Só me fala 👍